



VIVEKANANDA GLOBAL UNIVERSITY, JAIPUR



(Established by Act11/2012ofRajasthanGovt.Coveredu/s2(f)ofUGCAct,1956)

Lecture Notes

Course Code - UGCSE201 Course Name - Data Structures

Module 2: Stacks, Queues and Lists

Contents

- Module 2. 1. Definition, Array based implementation of stacks
- Module 2. 2. Examples- Infix, postfix, prefix representation
- Module 2. 3. Applications- Mathematical expression Evaluation
- Module 2. 4. Queues-Array based implementation of Queues
- Module 2. 5. Circular implementation of Queues, Double ended queues.**
- Module 2. 6. Linked List- Singly linked Lists and Operations associated with it
- Module 2. 7. Stack and Queue Implementation Using linked list,
- Module 2. 8. Straight circular implementation of doubly linked Lists, Priority queues

2.5.1 Introduction to Circular Queues

Circular Queue is also a linear data structure, which follows the principle of FIFO(First In First Out), but instead of ending the queue at the last position, it again starts from the first position after the last, hence making the queue behave like a circular data structure.

Basic features of Circular Queue

In case of a circular queue, head pointer will always point to the front of the queue, and tail pointer will always point to the end of the queue.

Initially, the head and the tail pointers will be pointing to the same location, this would mean that the queue is empty.

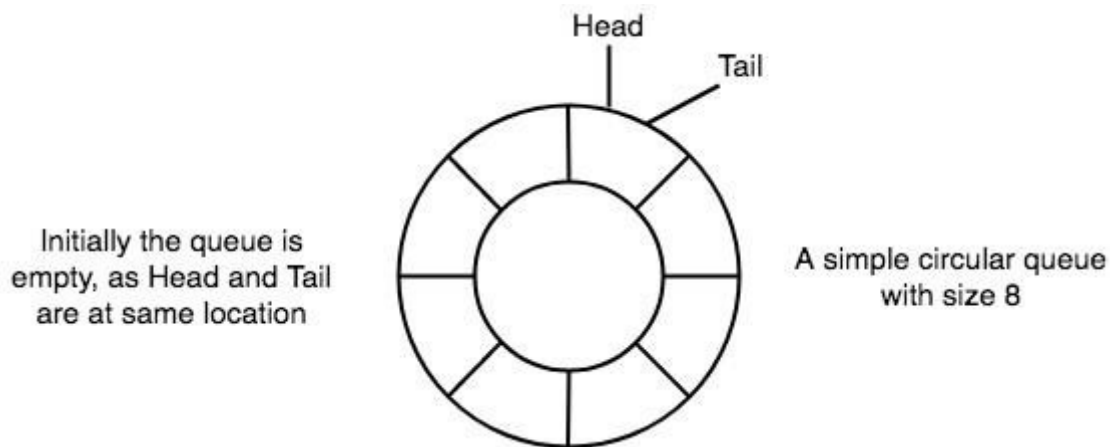


Figure 2.2: Circular Queue

New data is always added to the location pointed by the tail pointer, and once the data is added, tail pointer is incremented to point to the next available location.

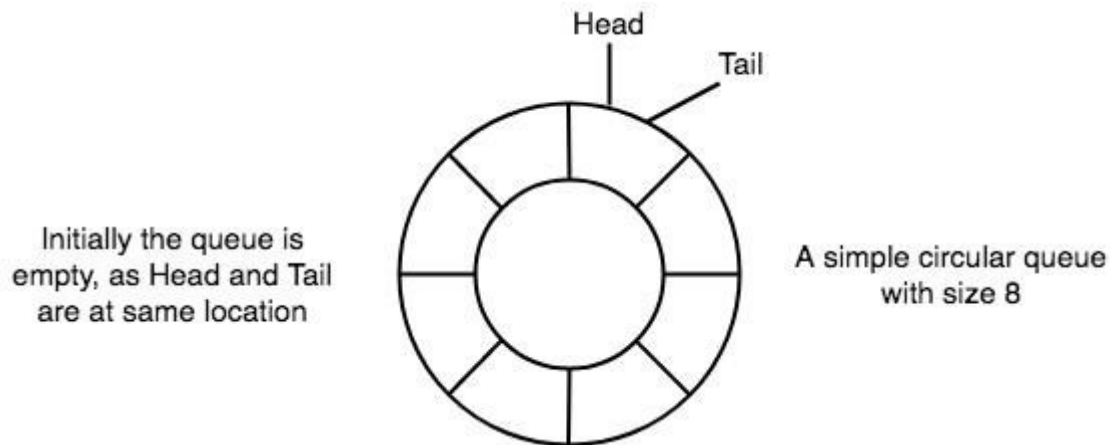


Figure 2.3: Circular Queue Insertion

In a circular queue, data is not actually removed from the queue. Only the head pointer is incremented by one position when dequeue is executed. As the queue data is only the data between head and tail, hence the data left outside is not a part of the queue anymore, hence removed.

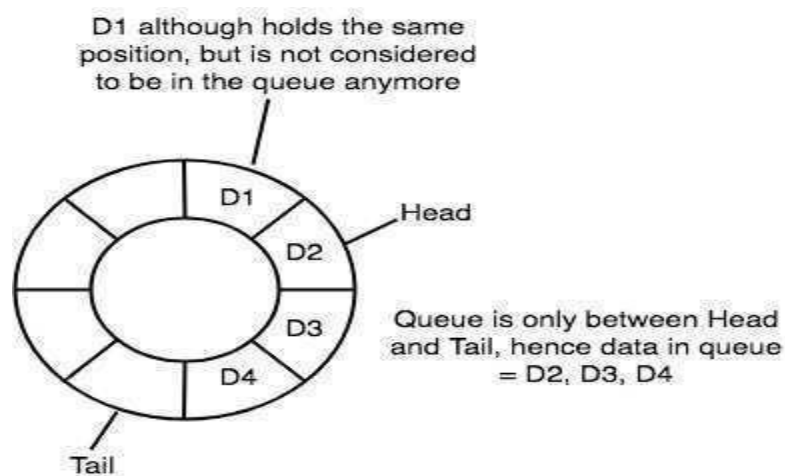


Figure 2.4: Circular Queue Insertion

The head and the tail pointer will get reinitialized to 0 every time they reach the end of the queue.

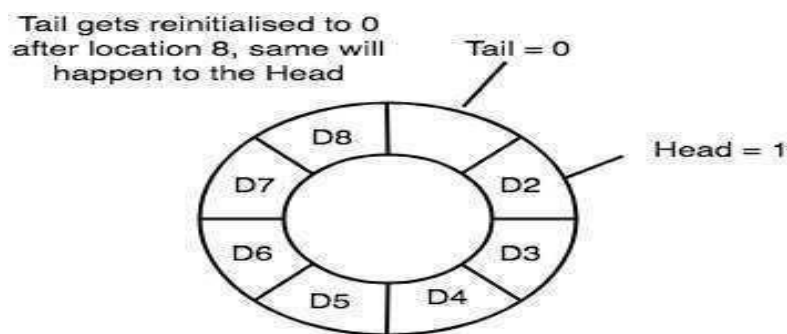


Figure 2.7: Circular Queue Insertion

Also, the head and the tail pointers can cross each other. In other words, head pointer can be greater than the tail. Sounds odd? This will happen when we dequeue the queue a couple of times and the tail pointer gets reinitialized upon reaching the end of the queue.

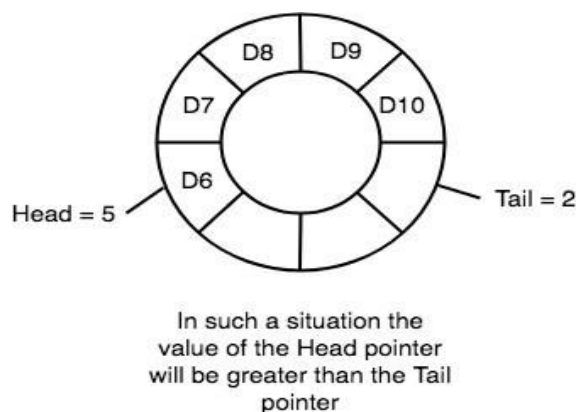


Figure 2.8: Circular Queue deletion Application of Circular Queue

- Computer controlled Traffic Signal System uses circular queue.
- CPU scheduling and Memory management.

CONCEPT OF DEQUE:

A deque, also known as a double-ended queue, is an ordered collection of items similar to the queue. It has two ends, a front and a rear, and the items remain positioned in the collection. New items can be added at either the front or the rear. Likewise, existing items can be removed from either end. In a sense, this hybrid linear structure provides all the capabilities of stacks and queues in a single data structure.

It is important to note that even though the deque can assume many of the characteristics of stacks and queues, it does not require the LIFO and FIFO orderings that are enforced by those data structures. It is up to you to make consistent use of the addition and removal operations.

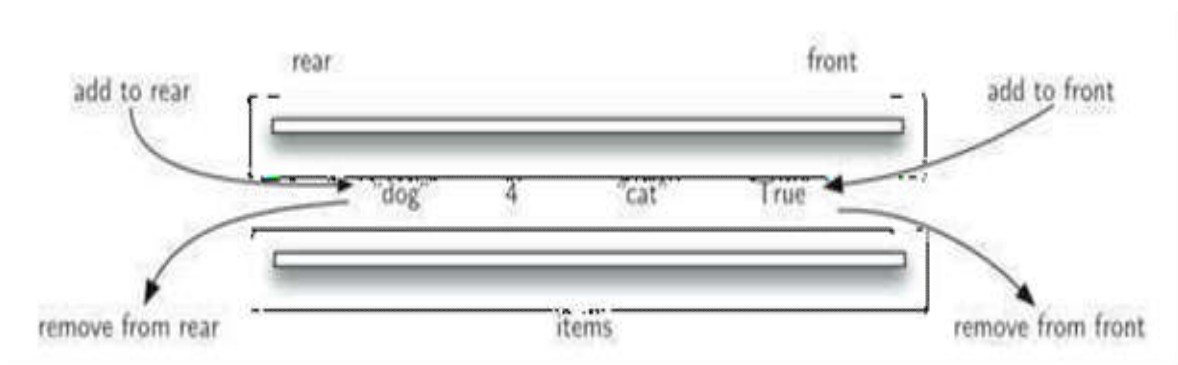


Figure 2.9: Deque

Operation on Deque:

- `insertFront()`: Adds an item at the front of Deque.
- `insertLast()`: Adds an item at the rear of Deque.
- `deleteFront()`: Deletes an item from front of Deque.
- `deleteLast()`: Deletes an item from rear of Deque.

In addition to above operations, following operations are also supported

- `getFront()`: Gets the front item from queue.
- `getRear()`: Gets the last item from queue.
- `isEmpty()`: Checks whether Deque is empty or not.
- `isFull()`: Checks whether Deque is full or not.

PRIORITY QUEUE:

Priority Queue is more specialized data structure than Queue. Like ordinary queue, priority queue has same method but with a major difference. In Priority queue items are ordered by key value so that item with the lowest value of key is at front and item with the highest value of key is at rear or vice versa. So we're assigned priority to item based on its key value. Lower the value, higher the priority. Following are the principal methods of a Priority Queue.

Basic Operations

- `insert / enqueue` – add an item to the rear of the queue.
- `remove / dequeue` – remove an item from the front of the queue.

Priority Queue Representation

We're going to implement Queue using array in this article. There is few more operations supported by queue which are following.

- `Peek` – get the element at front of the queue.
- `isFull` – check if queue is full.
- `isEmpty` – check if queue is empty.



VIVEKANANDA GLOBAL UNIVERSITY, JAIPUR



(Established by Act11/2012ofRajasthanGovt.Coveredu/s2(f)ofUGCAct,1956)

Insert / Enqueue Operation

Whenever an element is inserted into queue, priority queue inserts the item according to its order. Here we're assuming that data with high value has low priority.

Remove / Dequeue Operation

Whenever an element is to be removed from queue, queue get the element using item count. Once element is removed. Item count is reduced by one.

Applications of Deque:

Since Deque supports both stack and queue operations, it can be used as both. The Deque data structure supports clockwise and anticlockwise rotations in $O(1)$ time which can be useful in certain applications.

QUEUE SIMULATION:

Queuing simulation as a method is used to analyze how systems with limited resources distribute those resources to elements waiting to be served, while waiting elements may exhibit discrete variability in demand, i.e. arrival times and require discrete processing time.



VIVEKANANDA GLOBAL UNIVERSITY, JAIPUR



(Established by Act11/2012ofRajasthanGovt.Coveredu/s2(f)ofUGCAct,1956)

Queuing theory based analysis is regularly used for e.g. telecommunications, computer networks, predicting computer performance, traffic, call centres, etc.

APPLICATIONS OF QUEUES:

Queue is used when things don't have to be processed immediately, but have to be processed in First In First Out order like Breadth First Search. This property of Queue makes it also useful in following kind of scenarios.

- When a resource is shared among multiple consumers. Examples include CPU scheduling, Disk Scheduling.
- When data is transferred asynchronously (data not necessarily received at same rate as sent) between two processes. Examples include IO Buffers, pipes, file IO, etc.